

Test Scenarios for Nimbus Internal Editor story NNG-18106

Scenarios for Code Editor Functionalities:

1. File System Browsing in Sidebar:

- Verify that the sidebar displays the complete file directory structure.
- Ensure the user can navigate through folders and view files.
- Validate error handling for inaccessible or restricted directories.
- Ensure that all types of files can be uploaded
- Ensure that the file size limit of 25MBs is enforced while uploading the file
- In the upload file modal need to test the UI components like frontend validations, placeholders reset functionality etc
- File delete functionality should be available for the user.
- Relevant permissions need to be tested for all the buttons in the Internal Editor in different states of the UDF i.e when the UDF is in Updated state, in Under review, Approved and Rejected states.
- Test by uploading and not saving files, need to check if changes persist
- Test for UI support for long file/folder names, result names the UI should not be misaligned. If the outputs are too many, then for the long list of outputs there should be a scroller refer to NNG-18809

2. Editing Multiple Files:

- Test whether multiple files can be opened in tabs without errors.
- Verify changes in one file do not overwrite or affect others.
- Ensure unsaved changes prompt a warning before closing the file.

3. Rendering Files in Different Ways:

- Test for ipynb files:
 - Validate that cells can be edited and executed independently.
 - Verify that the execution results are displayed below each cell.
- Test for code files (.py, etc.):
 - Ensure they are editable and executable.
- Test for images/charts:
 - Validate that they are displayed correctly but remain non-editable.
- Test for html files:
 - Verify the content can be edited.
- Test for csv files:
 - Ensure the content is editable, especially for data rows and columns.
- Test for unsupported file types:
 - Validate they are displayed as non-editable with proper error messages.

4. **Cell Execution in Notebook Files:**
 - Verify that individual cells execute successfully and display accurate results.
 - Test for dependency errors when executing a cell dependent on prior cell outputs.
5. **Git Commit List:**
 - Validate that all commits for the repository are listed accurately.
 - Check if commit timestamps and messages are displayed correctly.
6. **Git Commit Functionality:**
 - Test committing changes to the repository with valid and invalid commit messages.
 - Validate if commit is successful for edited files.
7. **Uploading Code Files:**
 - Verify upload functionality for ipynb and py files within the size limit (max 2MB).
 - Test error handling for non-supported formats and files larger than 2MB.
 - Validate whether uploaded files appear correctly in the editor.

Scenarios for Code Execution Engine (CEE):

1. **Code Execution from Editor:**
 - Validate that ipynb files execute seamlessly with results displayed as required.
 - Test execution of py files with proper stdout and stderr outputs.
2. **Virtual Environment Initialization:**
 - Verify that a virtual environment is created for each session.(We can test this by installing libraries in one session and testing their presence in another session)
 - Test environment isolation by running concurrent sessions.
3. **Cloning Code from Version Controller:**
 - Check if the latest code is fetched from the version controller.
4. **Dependency Installation:**
 - Test installation of dependencies, including ipykernel, before code execution.
 - Validate error handling for missing or unsupported dependencies.
5. **IPython Kernel Initialization:**
 - Verify the kernel client initializes correctly for executing notebook cells.
 - Test kernel errors when executing code.
6. **Resource Limits Enforcement:**
 - Validate CPU and memory limits are applied during execution.(Need to check what are the limits)

- Test scenarios where resource limits are exceeded and ensure errors are returned.
- 7. **Execution Timeout:**
 - Validate timeout enforcement by running an infinite loop.
 - Ensure timeout errors are returned appropriately.
- 8. **Execution Results:**
 - Test for accuracy of execution results such as text, datasets, models, tables, charts, and images.
 - Validate retrieval of artifacts post-execution.
 - Test if these results are getting stored along with UDF files.
- 9. **Error Handling:**
 - Test for proper error messages and tracebacks on code failures.
 - Validate that execution failures are logged and displayed correctly.
- 10. **Resource Cleanup:**
 - Verify resources (e.g., memory, virtual environments) are freed after execution.
 - Test cleanup for canceled or timeout sessions.

Scenarios for Integration with Version Controller:

1. **Fetching Code:**
 - Test fetching the latest code before execution.
 - Validate error handling for version controller connectivity issues.
2. **Committing Changes:**
 - Verify committing edits from the editor to the repository.
 - Validate commit history updates after code changes.

Scenarios for User Session Management:

1. **Concurrent Sessions:**
 - Validate that multiple user sessions are supported without conflicts.
 - Verify session isolation, ensuring users cannot access others' environments.
2. **Closing Sessions:**
 - Test graceful session closure, ensuring no data loss.
 - Validate cleanup of unused virtual environments after inactivity.

Scenarios for WebSocket and Real-Time Updates:

1. **Real-Time Communication:**
 - Validate real-time updates between the editor and execution engine.
 - Test scenarios for delayed or dropped WebSocket connections.
2. **Execution Status Updates:**
 - Verify status updates (e.g., in progress, completed, failed) are sent in real-time.
3. **File Directory Events:**
 - Test real-time updates for file changes (e.g., new files or folders added/deleted).

Scenarios for Concurrent User Functionality:

Same UDF:

1. UDF Locking:

- Verify that if two users are trying to use the same UDF simultaneously, it gets locked for the second user.
- Test if User 1 is in **run mode**, the UDF should be locked for User 2.
- Test if User 1 is in **edit mode**, the UDF should be locked for User 2.
- Test if User 1 is in the **internal editor**, the UDF should be locked for User 2.

Different UDF:

2. Independent Collaboration:

- Verify that two collaborators can use different UDFs independently without conflict.
- Validate that both users' changes are saved along with proper tracking of individual edits.

3. UDF Manager Updates:

- On the UDF Manager landing page, verify that the 'last updated by' column gets updated correctly when edits are made using the internal editor.

4. Version History Updates:

- Test that changes made through the internal editor are reflected accurately in the Version History page.

Scenarios for allied Nimbus functionalities:

1. Sharing Cloning

- a. UDFs should be shared and cloned as expected and the Internal Editor should display the code.
- b. UDFs should get shared and cloned with pipelines
- c. UDFs should get shared and cloned with projects

2. GitHub Integration

- a. When we import a UDF via GitHub/BitBucket the imported repo and files should be visible in the internal editor

3. UDF Agent

- a. When we create a UDF locally via Agent the ipynb file should be visible in Internal Editor
- b. When we upload the results from the local, the result files should be rendered on the Editor UI
- c. If any files are not supported then those files should show an error or/and an infographic icon should be shown if user tries to open the file.

4. Logs

- a. Audit logs for save actions should be visible.

5. UDF Manager – Version History

- a. Version history should be visible when we save the changes from Internal Editor
 - b. Version history should be visible for changes in other files too
 - c. Only two types of entries should be visible here, one from agent commits and one from Internal editor commits
6. **Need to test the review request mechanism and related permissions with Internal Editor**
 7. **Help section should open from Internal Editor**
 8. **Help section content should be updated according to new Internal Editor**
 9. **Debug permissions should be removed from Admin Panel permission matrix and from create Collaborative projects form on UI**

Scenarios for Agent Functionality:

1. **Local Execution Without Locking:**
 - Verify that when the agent library is installed locally and a UDF is cloned onto the local machine, it can be executed, edited, and updated without locking the UDF on the server.
2. **Outdated Changes Validation:**
 - Test the scenario where:
 - User 1 has the UDF open in **edit mode**.
 - User 2 clones the UDF locally, makes changes, and commits them.
 - Validate that User 1 receives a warning indicating their changes are outdated.
 - Verify that if User 1 saves their changes, it overwrites the changes made by User 2.
 - If code is edited and saved in internal editor, then agent user should get a warning of conflict.
3. **Internal Editor Behaviour:**
 - Test the above "outdated changes validation" scenario for the internal editor and ensure the same workflow applies.

Scenarios for Non-Functional Requirements:

1. **Performance:**
 - Verify virtual environments are initialized within <10 seconds.
 - Test system behavior under parallel execution of 50 concurrent sessions.
2. **Security:**
 - Validate sandboxed execution environments to restrict unauthorized access.
 - Test authentication enforcement before code execution.
3. **Availability & Reliability:**
 - Verify auto-restart functionality for failed processes.
 - Test retry mechanisms for failed execution requests.

